Pokédex Retrô — Parte Teórica

Web Development Front-End · UniFECAF · Aluno: Sidney Albuquerque da Silva · Repositório: github.com/sidneysilva648/front-end · Site: sidneysilva648.github.io/front-end

1. O que é uma API e como funciona o consumo de dados em tempo real

API (*Application Programming Interface*) é um conjunto de regras que permite que um programa peça dados ou serviços a outro. Na web, as APIs mais comuns seguem o modelo **REST**: cada recurso tem um endereço (URL, chamado de *endpoint*) e o cliente faz uma requisição HTTP para ele. O servidor responde com um código de status (200 = sucesso, 404 = não encontrado) e um corpo de dados, normalmente em **JSON**, um formato de texto que o JavaScript converte facilmente em objetos.

O consumo é "em tempo real" porque o site não guarda os personagens no próprio código: toda vez que a página é aberta ou um filtro é alterado, o navegador faz novas requisições e monta a tela com o que a API devolve naquele momento. Se a PokéAPI atualizar um dado, o site exibe a versão nova sem nenhuma mudança no HTML. Essas requisições são **assíncronas**: a página continua respondendo enquanto espera a resposta, e por isso o Pokédex Retrô mostra cards de carregamento até os dados chegarem.

2. Manipulação do DOM e criação de elementos dinâmicos

O **DOM** (*Document Object Model*) é a representação do HTML em forma de árvore de objetos que o navegador cria ao carregar a página. Cada tag vira um nó que o JavaScript pode ler, alterar, criar ou remover. *DOM manipulation* é justamente usar o JavaScript para modificar essa árvore depois que a página já foi carregada, e o navegador redesenha a tela automaticamente.

No projeto, o `index.html` contém apenas um container vazio (`<div id="grade-cards">`). Para cada Pokémon recebido da API, o arquivo `cards.js` cria os elementos com `document.createElement`, preenche o texto com `textContent`, define imagem e atributos, encaixa uns dentro dos outros com `appendChild`/`append` e, por fim, insere o card pronto na grade. Usar `textContent` em vez de `innerHTML` garante que nenhum texto vindo da API seja interpretado como código HTML, o que é uma boa prática de segurança.

3. Funções utilizadas no consumo da API

Função / recurso	O que faz no projeto
`fetch(url)`	Envia a requisição HTTP GET para a PokéAPI e devolve uma *Promise* com a resposta (api.js, função buscarJSON).
`.then(callback)`	Executa o código quando a Promise é resolvida: verifica resposta.ok e lê o corpo com resposta.json().
`resposta.json()`	Converte o texto JSON da resposta em objeto JavaScript.
`.catch()` / `try...catch`	Trata falhas de rede ou status de erro, exibindo uma mensagem ao usuário em vez de quebrar a página.
`async` / `await`	Forma mais legível de esperar Promises; usada para organizar o fluxo em buscarDetalhesPokemon e iniciar.
`Promise.all` / `allSettled`	Dispara várias requisições em paralelo (dados + espécie, ou os 24 cards de uma página) e espera todas.
`document.createElement`	Cria cada elemento do card: article, h3, img, ul, li, dl, button. Para o ícone SVG do coração, createElementNS.
`appendChild` / `append`	Coloca um elemento dentro de outro, montando a estrutura do card e inserindo-o na grade.
`replaceWith` / `replaceChildren`	Troca o card de carregamento pelo card real e limpa a grade quando um filtro muda.

Função / recurso	O que faz no projeto
`addEventListener`	Reage a digitação na busca, mudança de filtros, clique no coração, em "Carregar mais" e em "Sortear outro".
`localStorage`	Guarda os números dos Pokémon favoritos no navegador, para que continuem marcados ao voltar ao site.

```
function buscarJSON(url) {
  return fetch(url).then((resposta) => {
    if (!resposta.ok) throw new Error(`Status ${resposta.status}`);
    return resposta.json();
  });
}
const card = document.createElement("article");
card.appendChild(document.createElement("h3")).textContent = "Pikachu";
grade.appendChild(card);
```

4. Motivo da escolha da API

O tema do trabalho são desenhos de sucesso dos anos 90 e 2000, e Pokémon é um dos maiores exemplos dessa época: o primeiro jogo saiu em 1996, o anime estreou em 1997 no Japão e chegou ao Brasil em 1999. Por isso o site foca nas quatro primeiras gerações (Pokémon nº 1 a 493, de 1996 a 2006).

Do ponto de vista técnico, a **PokéAPI** foi escolhida por quatro motivos:

- É gratuita e aberta, sem cadastro nem token, então o projeto funciona publicado no GitHub Pages sem expor nenhuma chave secreta (diferente da API da Marvel, que exige chave e assinatura).
- É estável e bem documentada, com imagens oficiais em alta resolução e suporte a CORS, permitindo chamadas direto do navegador.
- Tem dados ricos para os cards: tipos, status, medidas, categoria e descrição da Pokédex.
- Permite filtros reais com dados da própria API, como a lista de tipos e os Pokémon de cada tipo.

5. Regras de acesso e documentação da PokéAPI

Regra	Como funciona
Endereço base	`https://pokeapi.co/api/v2/` — documentação em pokeapi.co/docs/v2
Autenticação	Nenhuma. Não precisa de cadastro, chave ou token; todos os recursos são abertos.
Métodos	Somente leitura (HTTP GET). A API não permite criar, alterar ou apagar dados.
Limite de requisições	A limitação por quantidade de chamadas foi removida, mas a **política de uso justo** pede para não exagerar na frequência e para **guardar as respostas em cache**. Quem fizer abuso (ex.: ataque de negação de serviço) pode ter o IP bloqueado. O projeto respeita isso com um cache em memória (Map) que evita repetir a mesma URL.
Paginação	Listas retornam até 20 itens por padrão; os parâmetros `limit` e `offset` mudam a página. O site usa `limit=493` para obter as 4 gerações em uma chamada.
Formato	JSON, com CORS liberado (`Access-Control-Allow-Origin: *`). As imagens vêm de URLs do repositório de sprites no GitHub.
Idiomas	Os textos têm várias traduções, mas não há português. O site usa espanhol e, na falta dele, inglês.

Estrutura de resposta. Uma lista traz o total e os links de paginação; um recurso individual traz os dados detalhados (trecho resumido):

```
GET /pokemon?limit=2
{
  "count": 1351,
  "next": ".../pokemon?offset=2...",
  "previous": null,
  "results": [
    { "name": "bulbasaur",
      "url": ".../pokemon/1/" }, ...
  ]
}
```

```
GET /pokemon/25  (trecho)
{
  "id": 25, "name": "pikachu",
  "height": 4, "weight": 60,
  "types": [{ "type": { "name": "electric" } }],
  "stats": [{ "base_stat": 35, "stat": { "name": "hp" } }, ...],
  "sprites": { "other": { "official-artwork":
      { "front_default": "https://.../25.png" } } }
}
```

Altura e peso vêm em decímetros e hectogramas, por isso o código divide por 10 para mostrar metros e quilos. A descrição e a categoria ficam em outro endpoint, `/pokemon-species/{id}`, buscado em paralelo com `Promise.all`.

6. Resultado

Página publicada consumindo a API: os cards abaixo não existem no HTML; foram criados pelo JavaScript a partir das respostas da PokéAPI.

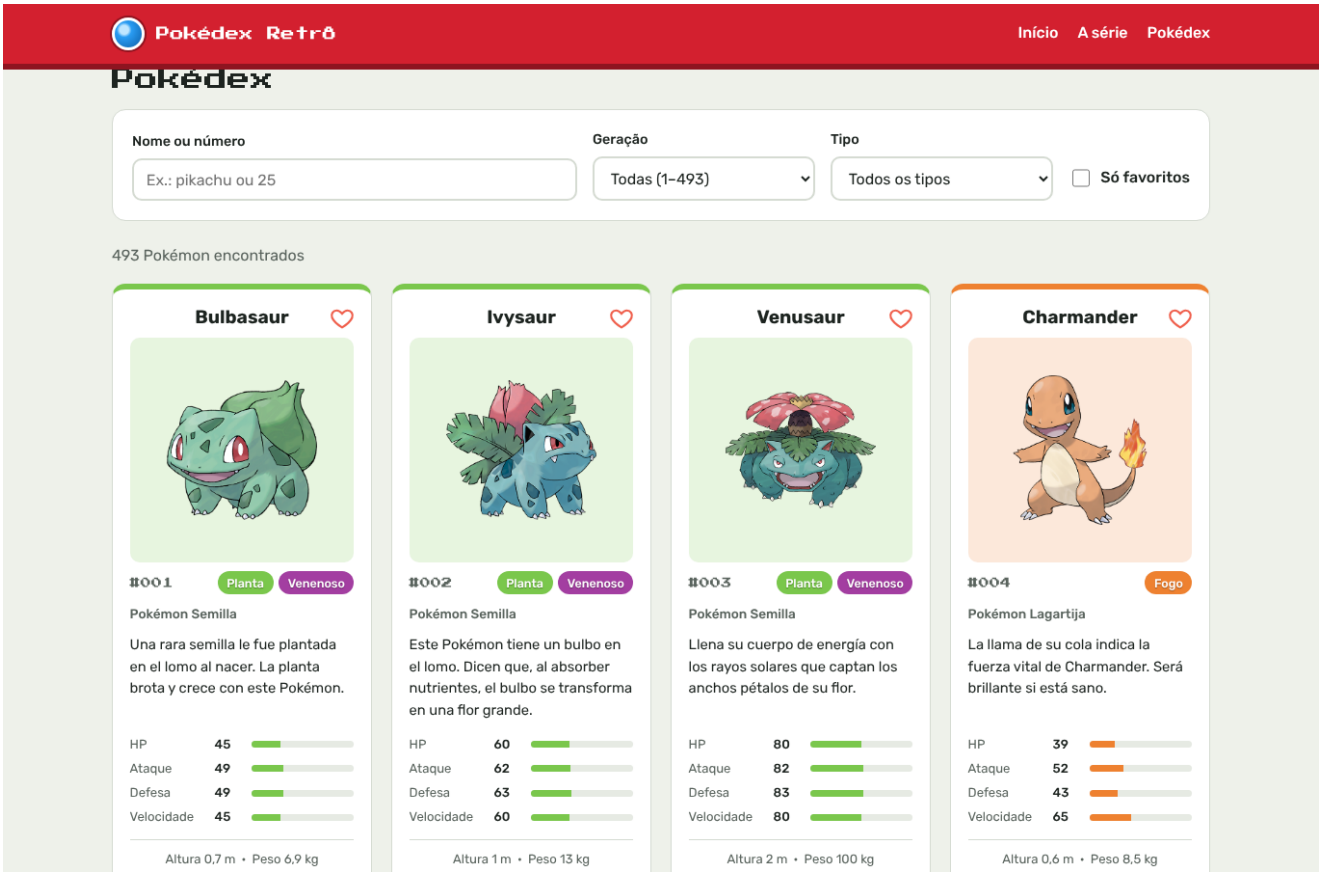


Figura 1 — Seção Pokédex com filtros e cards gerados dinamicamente